

Final Project Documentation

Date	04 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

1. INTRODUCTION

1.1 Project Overview

The *Personalized Networking Assistant* is an AI-powered web application designed to alleviate social anxiety and bridge domain-knowledge gaps during networking sessions. The solution parses event descriptions, extracts top conceptual themes using a local zero-shot classifier, and generates contextualized conversation icebreakers tailored to the user's specific goals. In addition, a fact-checking utility powered by public Wikipedia REST APIs provides instant reference checks on unfamiliar terms.

1.2 Purpose

Large professional conferences present substantial collaboration opportunities, yet attendees often fail to build connections due to communication barriers or unfamiliarity with domain buzzwords. This application automates starter generation and quick fact lookup in a single dashboard to empower attendees during offline events.

2. IDEATION PHASE

2.1 Problem Statement

A conference attendee (student/professional) wants to start meaningful, context-relevant conversations and verify buzzwords, but faces severe anxiety and lack of preparation, which makes them feel awkward and isolated during brief social gaps.

2.2 Empathy Map Canvas

SAYS 'I want to meet collaborators.' 'How do I start a discussion?'	THINKS 'Will I sound incompetent?' 'I wish I knew this tech domain.'
DOES - Stays alone near coffee tables. - Avoids direct contact, checks phone.	FEELS Socially isolated, anxious, awkward.

2.3 Brainstorming

The team evaluated multiple ideas. The final concept was chosen based on high execution feasibility and the value of combining zero-shot theme extraction, text generation, and third-party fact checks in a single client interface.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage of Journey	Discovery	Scan Description	Action & Remediation
Actions	Attendee opens the application and fills user bio details.	Attendee types the event text and personal interests.	Attendee clicks generate to view starters, checks wiki terms, and leaves feedback.
Touchpoints	Streamlit UI panel	FastAPI /analyze-event router	Conversation result display & thumbs buttons
Feelings	Skeptical, hoping for guidance.	Focused on target goals.	Relieved, confident to network.

3.2 Solution Requirement

- **FR-1:** Must perform zero-shot classification on event text to retrieve top 3 themes.
- **FR-2:** Must generate exactly 3 conversation icebreakers leveraging GPT-2.
- **FR-3:** Must query Wikipedia REST endpoints safely and return fallback definitions on 404.
- **NFR-1:** Latency under concurrent REST API calls must be less than 2.0s.
- **NFR-2:** Storage must run locally (JSON files) to enforce strict user data privacy.

3.3 Data Flow Diagram

Data flows from the Streamlit UI to the FastAPI orchestrator. The orchestrator routes payloads to the DistilBERT theme extraction service, passes results to the GPT-2 text-generator, and logs transaction files on disk.

3.4 Technology Stack

Component Layer	Technology Chosen	Justification
	Streamlit (Python)	Enables rapid layout prototyping and supports reactive state management.
	FastAPI / Uvicorn	High concurrency asynchronous web framework with automatic Swagger docs.
	DistilBERT / GPT-2	Offline-capable local models that run efficiently on standard CPU memory pools.
	JSON Flat Files	Lightweight files on disk, ensuring 100% user data privacy.

4. PROJECT DESIGN

4.1 Problem Solution Fit

The fit canvas matches user pains (social anxiety, lack of topics) directly with features (zero-shot theme parsing, automatic sentence completion, real-time Wikipedia fact verification widgets). Local hosting resolves connectivity limitations within busy event halls.

4.2 Proposed Solution

The proposed solution implements a two-tier application package. The FastAPI gateway exposes REST endpoints wrapping HuggingFace pipelines, which are loaded and cached globally at startup. The Streamlit frontend displays input panels, generated results, fact checking lookups, and transaction logs on the same page.

Resource Type	Specification Details	Allocation
Hardware	Intel i3/i5 or higher, 4GB RAM (8GB recommended), 10GB space.	Local system
Software Tools	Python 3.10+, FastAPI, Streamlit, Git, VS Code.	Local installation
Credential Keys	Google Gemini API Key (Reserved for future LLM upgrades).	Config env files

4.3 Solution Architecture

Our architecture uses a strict Presentation, Business API, and Local Data tiering structure. The Presentation Layer (Streamlit) sends HTTPS POST payloads containing event text to the FastAPI Server. The backend performs schema validation and invokes the core NLP modules to fetch classification results and generate conversation prompts.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Sprint	Epic Area	Tasks / User Stories	Estimation
Sprint-1	Setup & Design	Evaluate models (DistilBERT vs BART), define folders.	3 Points
Sprint-2	Backend Development	Build analysis, prompt generator, and wiki query modules.	5 Points
Sprint-3	Frontend UI & Test	Develop Streamlit layout, run unit tests via PyTest.	4 Points

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

S.No	Performance Metric	Target Threshold	Observed Value	Status
1	API Latency (/generate)	< 2.0 seconds	0.85 seconds	Pass
2	Wikipedia Fetch Latency	< 5.0 seconds	2.10 seconds	Pass
3	RAM Memory Usage	< 80% (System)	1.2 GB flat	Pass
4	Test Suite Pass Rate	100% coverage	8/8 Passed	Pass

7. RESULTS

7.1 Output Screenshots

- **Event Description Panel:** Text area to enter description details and user interests. Displays extracted themes in green sub-headers.
- **Starters Listing View:** Automatically prints 3 generated icebreakers based on extracted themes.
- **Wikipedia Lookup Box:** Text bar returning article summary in a clean banner.
- **Auditing History Pane:** Displays the last 5 logs and feedback votes from flat files.

8. ADVANTAGES & DISADVANTAGES

Advantages:

- **Data Privacy:** Offline models run locally, meaning no user interaction data is sent to external AI servers.
- **Offline Execution:** Operates without internet connectivity inside crowded venues.
- **Zero Cost:** Eliminates commercial LLM token costs.
- **Fast Startup:** Model caching ensures rapid classification under 1s.

Disadvantages:

- **Hardware Dependency:** Requires 4GB free RAM to cache model weights on the system host.
- **Limited GenAI Capability:** GPT-2 Small outputs can be simple and repetitive compared to large cloud LLMs.

9. CONCLUSION

The Personalized Networking Assistant successfully maps dynamic conference topics to personalized conversation starters. By coupling local zero-shot classifiers and transformers with fact check engines, it resolves both social hesitation and domain vocabulary gaps, establishing a secure, scalable tool for professional networking.

10. FUTURE SCOPE

- **Cloud Transition:** Enable serverless container hosting for scale.
- **Advanced LLM Integration:** Upgrade generative pipeline using Google Gemini API keys to retrieve highly complex responses.
- **Quantized execution:** Convert models to ONNX to double local execution speeds on older hardware.

11. APPENDIX

- **Source Code:** Backend logic in /app package, Frontend layout in /frontend/streamlit_app.py.
- **Dataset Link:** HuggingFace zero-shot mnli and gpt2 small weight datasets.

- **GitHub Link:** <https://github.com/Srikar-jayanthi/Personalized-Networking-Assistant.git>
- **Demo URL:** <https://drive.google.com/file/d/15cSu-8OCQ0fYzBHxKt87Ndb4CIBhJIHx/view?usp=sharing>